



Trabajo Práctico N°4: Cancelación Adaptiva de Ruido con Filtro LMS

Profesores:

Parlanti Gustavo Fernand (Presidente)

Molina German Alcides (Vocal)

Rossi Robreto Raul (Vocal)

Alumnos:

Campos

Testa

Verstraete

Mayo de 2026

Resumen

Realizar un programa aplicativo que implemente un sistema de cancelación adaptiva de ruido (ANC) sobre el microcontrolador STM32F446RE, utilizando un filtro LMS (Least Mean Squares) para adaptar dinámicamente los coeficientes de un modelo FIR que simula la trayectoria del ruido. El sistema debe procesar en tiempo real señales de entrada adquiridas mediante el ADC, generar una referencia de ruido blanco, colorearla mediante un filtro FIR de contaminación y recuperar la señal limpia mediante la cancelación adaptiva en un buffer configurable de 512 muestras. Los resultados se transmitirán a través de dos canales DAC: el primero contendrá la señal limpia recuperada $e(n) \approx s(n)$, mientras que el segundo contará la señal ruidosa $d(n) = s(n) + x_{\text{contam}}(n)$ para monitoreo.

Índice

1. Introducción	3
1.1. Objetivos del proyecto	3
2. Marco teórico	4
2.1. Cancelación Adaptiva de Ruido (ANC)	4
2.2. Filtro LMS (Least Mean Squares)	4
2.3. Generador de Ruido: LCG (Linear Congruential Generator)	4
2.4. Filtro FIR de Contaminación	5
3. Análisis de un proceso adaptativo	5
3.1. Implementación del algoritmo LMS	5
3.2. Resultados experimentales	6
4. Análisis de la influencia del ruido	9
4.1. Placa de Desarrollo	9
4.1.1. Microcontrolador STM32F446RE	9
4.1.2. Convertidor Analógico-Digital (ADC1)	10
4.1.3. Convertidor Digital-Analógico (DAC)	10
4.1.4. Timer 2 (TIM2)	10
4.1.5. Acceso Directo a Memoria (DMA2)	11
4.2. Arquitectura y Diseño del Sistema	11
4.2.1. Cadena de Procesamiento	11
4.2.2. Procesamiento por Bloques	11
4.2.3. Parámetros del Filtro LMS	11
4.3. Implementación	12
4.3.1. Inicialización del Sistema	12
4.3.2. Inicialización del FIR de Contaminación	12
4.3.3. Inicialización del Filtro LMS	12
4.3.4. Generación de Ruido: LCG	12
4.3.5. Procesamiento Principal: Buffer Half	13
4.3.6. Callbacks de Interrupción	13
4.4. Resultados Experimentales	14
5. Conclusiones	14

1. Introducción

La cancelación de ruido constituye un desafío fundamental en procesamiento de señales, especialmente en aplicaciones de comunicaciones, audio y sistemas de medición. Mientras que los filtros digitales tradicionales se basan en características espectrales conocidas del ruido, la cancelación adaptiva de ruido (ANC) ofrece una solución dinámica que se ajusta en tiempo real a cambios en las características del ruido.

En este trabajo práctico se implementa un sistema ANC sobre la plataforma STM32F446RE, continuando con la arquitectura desarrollada en laboratorios anteriores. El sistema utiliza un filtro LMS (Least Mean Squares), uno de los algoritmos adaptativos más utilizados por su estabilidad y eficiencia computacional. La arquitectura empleada sigue un modelo de “feedforward” donde:

- Una referencia de ruido blanco se genera mediante un generador pseudo-aleatorio (LCG).
- Esta referencia se filtra con un FIR de contaminación que modela la trayectoria del ruido desde la fuente hasta la señal de interés.
- El ruido coloreado resultante se suma a una señal limpia para simular un escenario de contaminación.
- Un filtro LMS recibe la referencia limpia y la señal ruidosa, aprendiendo adaptativamente el modelo FIR.
- La salida del LMS estima el ruido, permitiendo cancelarlo y recuperar la señal limpia.

El presente documento describe la implementación completa del sistema, incluyendo la arquitectura de procesamiento, la configuración de periféricos, el algoritmo LMS y los resultados experimentales obtenidos.

1.1. Objetivos del proyecto

- Implementar un sistema de cancelación adaptiva de ruido (ANC) sobre el microcontrolador STM32F446RE.
- Generar una referencia de ruido blanco pseudo-aleatorio utilizando un generador congruencial lineal (LCG).
- Diseñar un filtro FIR que modele la trayectoria de contaminación del ruido, simulando cómo el ruido se propaga y filtra antes de llegar a la señal de interés.
- Implementar un filtro LMS adaptivo que aprenda dinámicamente los coeficientes del FIR de contaminación.
- Procesar señales en tiempo real adquiridas mediante el ADC, aplicando el algoritmo LMS en bloques de 256 muestras con doble buffering y DMA.
- Transmitir dos canales de salida mediante DAC: la señal limpia recuperada $e(n)$ y la señal ruidosa de entrada $d(n)$ para monitoreo.
- Analizar experimentalmente la convergencia del algoritmo LMS bajo diferentes condiciones operativas.
- Demostrar la efectividad del sistema en cancelar ruido coloreado mediante análisis espectral.

2. Marco teórico

2.1. Cancelación Adaptiva de Ruido (ANC)

La cancelación adaptiva de ruido es una técnica que utiliza información de una referencia de ruido para estimar y cancelar el ruido presente en una señal deseada. A diferencia de los filtros digitales convencionales que tienen coeficientes fijos, los filtros adaptativos actualizan sus coeficientes en tiempo real basándose en una regla de minimización de error.

El modelo estándar de ANC “feedforward” con estructura LMS puede representarse como:

$$d(n) = s(n) + x_c(n)$$

donde $d(n)$ es la señal deseada (señal limpia más ruido), $s(n)$ es la señal de interés y $x_c(n)$ es el ruido coloreado que contamina la señal.

El sistema cuenta con una referencia de ruido $x_{ref}(n)$ que está correlada con $x_c(n)$ pero sin la coloración. El objetivo del filtro adaptativo es estimar $x_c(n)$ minimizando el error:

$$e(n) = d(n) - y(n)$$

donde $y(n)$ es la salida del filtro adaptativo. Cuando el filtro converge, $y(n) \approx x_c(n)$ y $e(n) \approx s(n)$.

2.2. Filtro LMS (Least Mean Squares)

El filtro LMS es un algoritmo adaptativo que actualiza los coeficientes del filtro mediante el método del descenso por gradiente. La ecuación de actualización de coeficientes es:

$$\mathbf{w}(n+1) = \mathbf{w}(n) + \mu e(n) x_{ref}(n-k)$$

donde $\mathbf{w}(n)$ es el vector de coeficientes del filtro, μ es el paso de adaptación (step size), $e(n)$ es el error instantáneo y $x_{ref}(n-k)$ es el vector de entrada retardada.

El paso de adaptación μ controla la velocidad de convergencia: valores pequeños resultan en convergencia lenta pero estable; valores grandes pueden causar oscilaciones o inestabilidad. El criterio de estabilidad teórico es:

$$0 < \mu < \frac{1}{M \cdot E[x_{ref}^2(n)]}$$

donde M es el número de coeficientes del filtro y $E[x_{ref}^2(n)]$ es la potencia de la señal de referencia.

2.3. Generador de Ruido: LCG (Linear Congruential Generator)

Para obtener una referencia de ruido blanco pseudo-aleatorio sin requerir un periférico RNG hardware, se utiliza un generador congruencial lineal (LCG), definido como:

$$x_{n+1} = (a \cdot x_n + c) \text{ mód } m$$

En la implementación se utiliza $a = 1664525$, $c = 1013904223$ y $m = 2^{24}$, que generan una secuencia de buena calidad estadística.

La ventaja del LCG es que es determinístico y mantiene estado continuo entre bloques, generando una secuencia estacionaria sin discontinuidades.

2.4. Filtro FIR de Contaminación

El filtro FIR de contaminación modela la trayectoria del ruido desde la fuente hasta la señal de interés:

$$x_c(n) = \sum_{k=0}^{L-1} h(k) \cdot x_{ref}(n-k)$$

donde $h(k)$ son los coeficientes que definen el canal de propagación. Este filtro introduce tanto retardo como coloración espectral al ruido de referencia.

En la implementación, los coeficientes del FIR definen un filtro paso-banda (30 coeficientes) que simula una trayectoria acústica o electrónica realista.

3. Análisis de un proceso adaptativo

Como primera etapa del laboratorio se estudió el comportamiento de un filtro adaptativo basado en el algoritmo LMS (*Least Mean Squares*) bajo el esquema de identificación de planta. El objetivo de esta sección es analizar el proceso de adaptación de los coeficientes del filtro y evaluar la influencia de los principales parámetros del algoritmo, particularmente el paso de adaptación μ y las características de la señal de entrada, sobre la velocidad de convergencia y el error de identificación.

Para ello se desarrolló un programa en lenguaje C que implementa una planta FIR de 30 coeficientes con respuesta al impulso conocida y un filtro adaptativo de igual longitud encargado de identificar dicha planta. Durante la simulación se calcula el error cuadrático medio (MSE), promediado cada 100 muestras, y la diferencia entre los coeficientes de la planta y los coeficientes estimados por el filtro adaptativo, permitiendo observar cuantitativamente el proceso de convergencia.

Con el fin de analizar la influencia de la señal de excitación sobre el desempeño del algoritmo, se realizaron ensayos utilizando tanto una señal pseudoaleatoria como una señal constante a la entrada del sistema. Asimismo, para cada caso se evaluaron distintos valores del parámetro de adaptación μ , permitiendo comparar la velocidad de convergencia y el error residual alcanzado por el filtro.

Finalmente, se desarrolló un script en Python encargado de procesar los datos generados por la simulación y representar gráficamente la evolución temporal del MSE y del error de los coeficientes para cada una de las condiciones analizadas. Estas gráficas constituyen la base para el análisis experimental presentado en las secciones siguientes.

3.1. Implementación del algoritmo LMS

La implementación del algoritmo LMS se realizó en lenguaje C utilizando una estructura de identificación de planta. En primer lugar se define una planta FIR de longitud $L = 30$, cuyos coeficientes permanecen fijos durante toda la simulación y representan la respuesta al impulso del sistema que el filtro adaptativo debe identificar.

```

1      #define L 30
2
3      static const double h[L] = {
4          ...
5      };

```

El programa permite realizar ensayos con dos tipos de señales de entrada: una señal pseudoaleatoria distribuida uniformemente en el intervalo $[-1, 1]$ y una señal constante de amplitud configurable. Además, los principales parámetros de la simulación, como el valor de μ , el número de muestras procesadas, el tipo de entrada y el nombre de los archivos de salida, se reciben como argumentos desde la línea de comandos, permitiendo automatizar los ensayos para distintas condiciones de operación.

El núcleo del algoritmo se encuentra dentro del lazo principal de procesamiento. Para cada muestra se genera el valor de entrada, se actualiza el buffer con las últimas L muestras y se calculan simultáneamente la salida de la planta $d(n)$ y la salida del filtro adaptativo $y(n)$ mediante el producto interno entre los coeficientes y el vector de entrada.

```

1      for (int i = L - 1; i > 0; i--)
2          x_buffer[i] = x_buffer[i - 1];
3      x_buffer[0] = x;
4
5      double d = 0.0;
6      double y = 0.0;
7
8      for (int i = 0; i < L; i++) {
9          d += h[i] * x_buffer[i];
10         y += w[i] * x_buffer[i];
11     }

```

A partir de la diferencia entre la salida deseada y la salida del filtro se obtiene la señal de error

$$e(n) = d(n) - y(n),$$

la cual se emplea para actualizar los coeficientes del filtro adaptativo siguiendo la regla LMS

$$w_i(n+1) = w_i(n) + \mu e(n) x(n-i).$$

La implementación correspondiente se muestra en el siguiente fragmento de código.

```

1      double e = d - y;
2
3      for (int i = 0; i < L; i++) {
4          w[i] += mu * e * x_buffer[i];
5      }

```

Durante la simulación se calcula el error cuadrático medio (MSE), promediado cada 100 muestras, y el error RMS entre los coeficientes estimados y los coeficientes reales de la planta. Estas métricas, junto con la evolución temporal de los coeficientes, se almacenan en archivos .csv para su posterior procesamiento y representación gráfica mediante un script desarrollado en Python.

3.2. Resultados experimentales

Con el objetivo de evaluar el desempeño del algoritmo LMS se realizaron distintas simulaciones modificando el valor del paso de adaptación μ . Durante cada ensayo se registró la evolución temporal del error cuadrático medio (MSE), el error RMS entre los coeficientes de la planta y los coeficientes estimados, así como la evolución individual de cada coeficiente del filtro adaptativo.

La figura 1 muestra la evolución del error cuadrático medio para distintos valores de μ utilizando una señal de entrada pseudoaleatoria. En todos los casos puede observarse una disminución progresiva del error durante las primeras iteraciones, indicando que la salida del filtro adaptativo se aproxima progresivamente a la salida de la planta. Asimismo, se

verifica el compromiso característico del algoritmo LMS entre velocidad de convergencia y estabilidad: valores pequeños de μ producen una convergencia lenta pero estable, mientras que al incrementar este parámetro el tiempo necesario para identificar la planta disminuye considerablemente.

Cabe destacar que, para los mayores valores de μ , el MSE alcanza valores del orden de 10^{-32} , correspondientes al límite de precisión numérica de las variables de doble precisión utilizadas durante la simulación. Esto indica que el error de identificación es prácticamente nulo y que el filtro adaptativo reproduce con gran exactitud la respuesta de la planta.

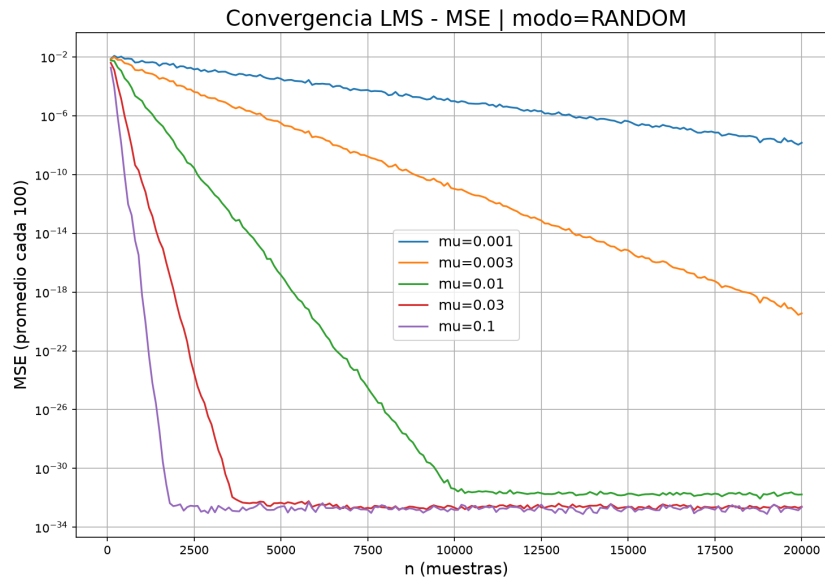


Figura 1: Evolución del MSE para distintos valores de μ utilizando una señal de entrada pseudoaleatoria.

La figura 2 presenta la evolución del error RMS entre los coeficientes estimados por el algoritmo y los coeficientes reales de la planta. Inicialmente el error es elevado debido a que todos los coeficientes del filtro adaptativo se inicializan en cero. A medida que avanza el proceso de adaptación, el error disminuye hasta estabilizarse alrededor de un valor muy próximo a cero, indicando que los coeficientes estimados convergen hacia los coeficientes reales de la planta.

Si bien el error RMS no alcanza exactamente el valor cero, las diferencias finales entre los coeficientes reales y estimados son del orden de la precisión numérica del sistema, por lo que pueden atribuirse a errores de redondeo inherentes a la representación en punto flotante. En contraste, la señal pseudoaleatoria posee un contenido espectral amplio que excita simultáneamente todos los coeficientes de la planta, permitiendo que el algoritmo LMS disponga de la información necesaria para reconstruir correctamente su respuesta al impulso.

Puede observarse además que el incremento del parámetro de adaptación μ reduce significativamente la cantidad de muestras necesarias para alcanzar la convergencia. Mientras que para $\mu = 0,001$ el algoritmo aún no converge completamente luego de 20000 muestras, para $\mu = 0,03$ y $\mu = 0,1$ la identificación se completa aproximadamente antes de las 4000 y 2000 muestras, respectivamente. Esto se debe a que un mayor valor de μ incrementa la velocidad de adaptación de los coeficientes, aunque valores excesivamente elevados podrían conducir a un comportamiento inestable del algoritmo.

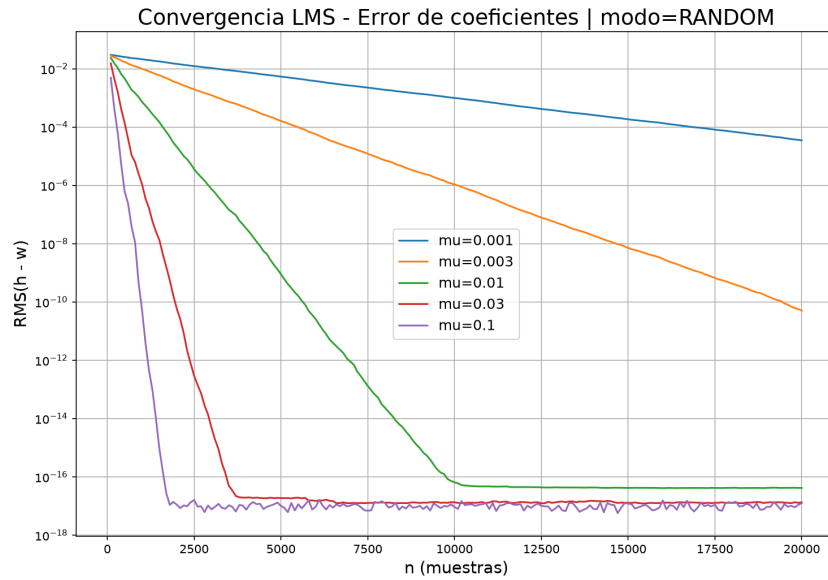


Figura 2: Evolución del error RMS de los coeficientes para distintos valores de μ .

Con el propósito de analizar la influencia de la señal de entrada sobre el proceso adaptativo, también se realizaron ensayos utilizando una señal constante de amplitud $x = 0,5$. Los resultados correspondientes se muestran en las figuras 3 y 4. En este caso se observa que el algoritmo no logra identificar correctamente la planta. Si bien durante las primeras iteraciones los coeficientes del filtro adaptativo presentan una ligera variación, tanto el MSE como el error RMS permanecen prácticamente constantes a lo largo de la simulación. Este comportamiento se debe a que una señal constante no constituye una señal persistentemente excitante, por lo que no aporta información suficiente para estimar de manera unívoca todos los coeficientes de la planta.

Este resultado coincide con la teoría de filtros adaptativos, ya que la convergencia del algoritmo LMS requiere que la matriz de autocorrelación de la señal de entrada tenga rango completo. Una señal constante genera una matriz singular, impidiendo la identificación de todos los coeficientes de la planta.

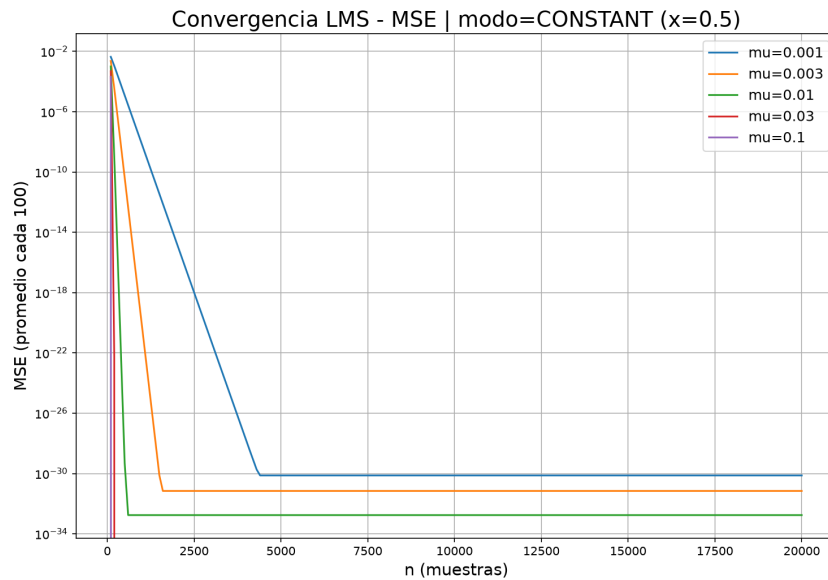


Figura 3: Evolución del MSE utilizando una señal de entrada constante ($x = 0,5$).

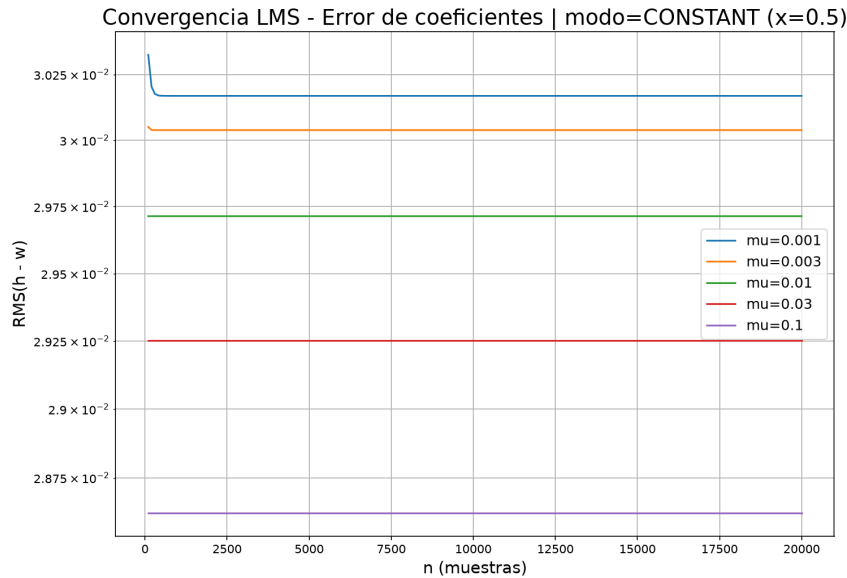


Figura 4: Evolución del error RMS de los coeficientes utilizando una señal de entrada constante ($x = 0,5$).

En conjunto, los resultados obtenidos demuestran el correcto funcionamiento del algoritmo LMS implementado. La disminución del MSE y del error RMS cuando se emplea una señal pseudoaleatoria confirma que el filtro adaptativo logra identificar correctamente la planta cuando la señal de entrada posee suficiente contenido espectral para excitar todos sus coeficientes. Además, los ensayos permiten verificar experimentalmente el compromiso existente entre rapidez de convergencia y estabilidad al variar el parámetro μ , así como la necesidad de utilizar señales persistentemente excitantes para garantizar la convergencia del algoritmo.

Los resultados obtenidos concuerdan con el comportamiento esperado para el algoritmo LMS. En particular, se verifica experimentalmente que la velocidad de convergencia depende directamente del paso de adaptación μ , mientras que la correcta identificación de la planta requiere una señal de entrada suficientemente excitante. Estos resultados validan tanto la implementación desarrollada como los fundamentos teóricos del algoritmo estudiados.

4. Análisis de la influencia del ruido

4.1. Placa de Desarrollo

4.1.1. Microcontrolador STM32F446RE

El STM32F446RE es un microcontrolador de alto rendimiento basado en ARM Cortex-M4 con:

- Núcleo: Cortex-M4 con soporte para FPU (unidad de punto flotante de 32 bits)
- Frecuencia de operación: Hasta 180 MHz
- Memoria Flash: 512 KB
- Memoria RAM: 128 KB
- Periféricos: ADC de 12 bits, DAC de 12 bits, múltiples timers, DMA, UART

En este proyecto se opera a 90 MHz (dividido del reloj máximo de 180 MHz) para mantener márgenes de estabilidad.

4.1.2. Convertidor Analógico-Digital (ADC1)

- Resolución: 12 bits
- Rango: 0 a 3.3V (mapeado a 0 a 4095 digital)
- Frecuencia de muestreo: 16 kHz
- Fuente de trigger: Salida TRGO de TIM2
- Modo: DMA en circular (double-buffering)
- Buffer: 512 muestras (256 por semibuffer para procesamiento)

La normalización a rango $[-1, +1]$ se realiza mediante:

$$s(n) = \frac{\text{ADC}(n) - 2048}{2047,5}$$

4.1.3. Convertidor Digital-Analógico (DAC)

- Resolución: 12 bits
- Rango: 0 a 3.3V
- Canales: 2 (CH1 para $e(n)$, CH2 para $d(n)$)
- Modo: DMA doble para transmisión continua
- Actualización: Sincronizada con ADC mediante TIM2

La conversión de punto flotante a 12 bits se realiza como:

$$\text{DAC}(n) = \lfloor (f(n) + 1,0) \times 2047,5 + 0,5 \rfloor$$

clampando el resultado entre 0 y 4095.

4.1.4. Timer 2 (TIM2)

- Configuración: Contador de 32 bits (máxima resolución)
- Función: Generar salida TRGO cada período para disparar ADC
- Frecuencia: 16 kHz
- Cálculo de período: $\text{ARR} = \frac{f_{CLK}}{f_{deseada}} - 1 = \frac{90 \times 10^6}{16 \times 10^3} - 1 = 5624$

4.1.5. Acceso Directo a Memoria (DMA2)

- Stream 0: ADC $\rightarrow$ buffer RAM (modo circular)
- Stream 6 y 7: RAM $\rightarrow$ DAC (modo circular para dual DAC)
- Modo: Circular con generación de interrupciones Half y Full Transfer
- Ancho de dato: 16 bits

El modo circular permite que DMA actualice continuamente los buffers sin intervención del CPU, minimizando latencia y jitter.

4.2. Arquitectura y Diseño del Sistema

4.2.1. Cadena de Procesamiento

La arquitectura del sistema implementa la siguiente cadena de señal:

1. **Generación de ruido:** Se genera una secuencia pseudo-aleatoria $x_{ref}(n)$ mediante LCG.
2. **Coloración del ruido:** El ruido se filtra con FIR(h_c) para obtener $x_c(n)$.
3. **Contaminación:** Se suma al signal limpio $s(n)$: $d(n) = s(n) + x_c(n)$.
4. **Estimación LMS:** El LMS recibe $x_{ref}(n)$ y $d(n)$, produciendo estimación $y(n) \approx x_c(n)$.
5. **Cancelación:** Se calcula $e(n) = d(n) - y(n) \approx s(n)$.
6. **Salida:** DAC1 transmite $e(n)$ (señal limpia), DAC2 transmite $d(n)$ (monitoreo).

4.2.2. Procesamiento por Bloques

El sistema utiliza doble buffering con buffers de 512 muestras. Cada semibuffer (256 muestras) se procesa tras la interrupción de DMA, evitando conflictos:

$$t_{procesamiento} = \frac{256}{16000} \approx 16 \text{ ms}$$

El marcador `processing_busy` asegura que no se procesen bloques simultáneamente.

4.2.3. Parámetros del Filtro LMS

- Número de coeficientes: $M_{LMS} = 60$
- Paso de adaptación: $\mu = 0,005$
- Criterio teórico: $\mu_{max} \approx \frac{1}{60 \times 0,03} \approx 0,556$
- Estabilidad: Conservadora ($\mu = 0,005' \mu_{max}$)
- Tiempo de convergencia estimado: ≈ 9 segundos a 16 kHz

4.3. Implementación

4.3.1. Inicialización del Sistema

```

1      /* Habilitar contador de ciclos DWT para profiling */
2      CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
3      DWT->CYCCNT = 0;
4      DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
5
6      /* Inicializar reloj del sistema a 90 MHz */
7      SystemClock_Config();
8
9      /* Inicializar periféricos */
10     MX_GPIO_Init();
11     MX_DMA_Init();
12     MX_ADC1_Init();
13     MX_TIM2_Init();
14     MX_DAC_Init();
15
16     /* Configurar TIM2 para 16 kHz exactos */
17     __HAL_TIM_SET_AUTORELOAD(&htim2, 5624);

```

4.3.2. Inicialización del FIR de Contaminación

```

1      /* Inicializar FIR de contaminación (30 coeficientes) */
2      arm_fir_init_f32(&fir_contam, CONTAM_TAPS,
3                      fir_contam_coeffs,
4                      fir_contam_state, BUFFER_SIZE / 2);
5
6      /* Los coeficientes modelan un filtro paso-banda
7      que simula la trayectoria acústica/electrónica */
8      static float32_t fir_contam_coeffs[30] = {
9          -0.0016f, -0.002f, -0.0012f, 0.0018f, 0.0068f, 0.0101f,
10         0.006f, -0.0084f, -0.0279f, -0.0381f, -0.0217f, 0.0305f,
11         0.1102f, 0.1918f, 0.2436f, 0.2436f, 0.1918f, 0.1102f,
12         0.0305f, -0.0217f, -0.0381f, -0.0279f, -0.0084f, 0.006f,
13         0.0101f, 0.0068f, 0.0018f, -0.0012f, -0.002f, -0.0016f
14     };

```

4.3.3. Inicialización del Filtro LMS

```

1      /* Inicializar LMS con 60 coeficientes
2      mu = 0.005 para convergencia estable */
3      arm_lms_init_f32(&lms, ADAPT_TAPS, lms_coeffs,
4                      lms_state, mu, BUFFER_SIZE / 2);
5
6      /* Los coeficientes arrancan en 0 y convergen
7      durante la operación del sistema */
8      float32_t mu = 0.005f;
9      float32_t lms_coeffs[60]; /* Inicializados a 0 */

```

4.3.4. Generación de Ruido: LCG

```

1      void generar_ruido(float32_t *x, uint32_t N,
2                          float32_t amplitud)
3      {
4          for (uint32_t i = 0; i < N; i++) {
5              /* Generador congruencial lineal */
6              rng_state = 1664525u * rng_state + 1013904223u;
7              uint32_t rnd = rng_state & 0x00FFFFFFu;
8
9              /* Normalizar a [-1, +1] */
10             float32_t u = (float32_t)rnd / 16777216.0f;
11             x[i] = amplitud * (2.0f * u - 1.0f);

```

```

12     }
13 }

```

4.3.5. Procesamiento Principal: Buffer Half

```

1  void process_buffer_half(uint16_t *adc_in,
2                          uint16_t *dac_out,
3                          uint16_t offset,
4                          volatile uint32_t *cycles)
5  {
6      /* Paso 1: Normalizar entrada ADC a [-1, +1] */
7      for (uint16_t i = 0; i < 256; i++) {
8          float32_t val = ((float32_t)adc_in[i + offset]
9                          - 2048.0f) / 2047.5f;
10         s_buffer[i] = (val > 1.0f) ? 1.0f :
11                      (val < -1.0f) ? -1.0f : val;
12     }
13
14     /* Paso 2: Generar referencia de ruido */
15     generar_ruido(x_ref_buffer, 256, 0.3f);
16
17     /* Paso 3: Colorear ruido con FIR */
18     arm_fir_f32(&fir_contam, x_ref_buffer,
19               x_contam_buffer, 256);
20
21     /* Paso 4: Contaminar señal limpia */
22     for (uint16_t i = 0; i < 256; i++) {
23         d_buffer[i] = s_buffer[i] + x_contam_buffer[i];
24     }
25
26     /* Paso 5: Salida DAC2 - señal ruidosa */
27     convert_float_to_dac_buffer(d_buffer,
28                               &dac_buffer2[offset], 256);
29
30     /* Paso 6: Aplicar LMS adaptativo */
31     arm_lms_f32(&lms, x_ref_buffer, d_buffer,
32               y_buffer, e_buffer, 256);
33
34     /* Paso 7: Salida DAC1 - señal limpia */
35     convert_float_to_dac_buffer(e_buffer,
36                               &dac_out[offset], 256);
37 }

```

4.3.6. Callbacks de Interrupción

```

1  /* DMA Half Transfer (primer semibuffer) */
2  void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc)
3  {
4      if (hadc->Instance != ADC1) return;
5      if (processing_busy) return;
6      processing_busy = 1;
7      process_buffer_half(adc_buffer, dac_buffer, 0,
8                          &proc_cycles_half);
9      processing_busy = 0;
10 }
11
12 /* DMA Transfer Complete (segundo semibuffer) */
13 void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc)
14 {
15     if (hadc->Instance != ADC1) return;
16     if (processing_busy) return;
17     processing_busy = 1;
18     process_buffer_half(adc_buffer, dac_buffer, 256,
19                         &proc_cycles_full);
20     processing_busy = 0;
21 }

```

4.4. Resultados Experimentales

Para evaluar el firmware implementado en la placa STM32, se inyectó una señal senoidal de 3,3 Vpp con una tensión de offset de 1,65 V. El ensayo se realizó para dos frecuencias de entrada, 100 Hz y 1 kHz, con el objetivo de comparar la respuesta del sistema ante distintas condiciones de operación.

En la Figura 5 se muestra la comparación para una señal de entrada de 100 Hz. La señal contaminada se observa en amarillo, mientras que la señal de salida limpia se representa en celeste.

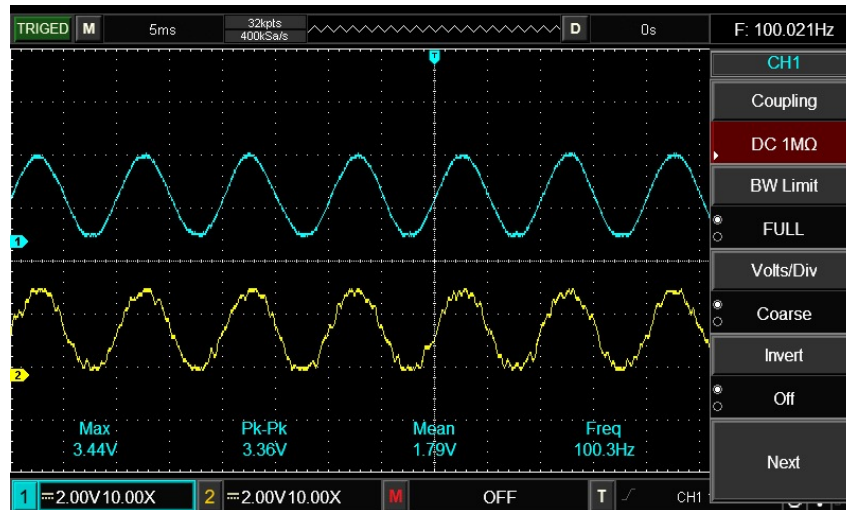


Figura 5: Salida limpia para señal de entrada de 100 Hz.

En la Figura 6 se presenta el mismo ensayo para una frecuencia de entrada de 1 kHz. Nuevamente, la señal contaminada corresponde al trazo amarillo, mientras que la salida limpia corresponde al trazo celeste.

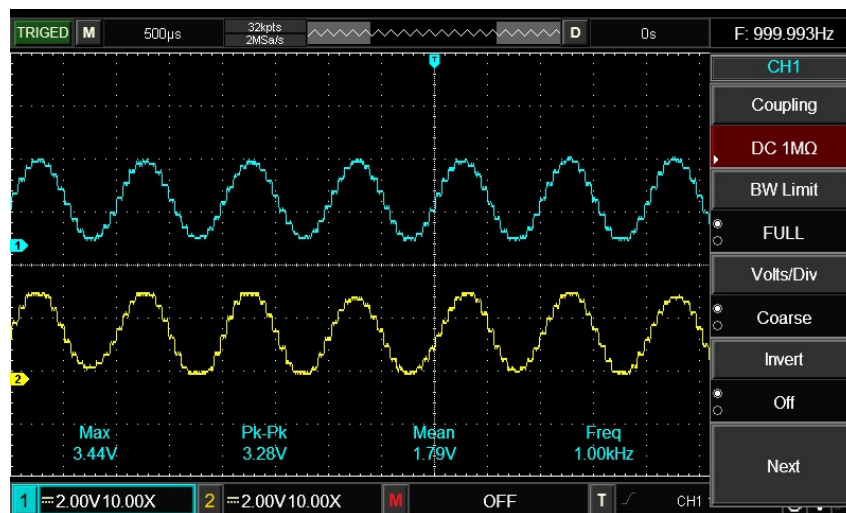


Figura 6: Salida limpia para señal de entrada de 1 kHz.

5. Conclusiones

En una primera etapa se analizó el comportamiento del algoritmo LMS bajo un esquema de identificación de planta, permitiendo verificar experimentalmente los principales

conceptos desarrollados durante el estudio teórico. Los resultados obtenidos mostraron que el parámetro de adaptación μ determina el compromiso entre velocidad de convergencia y estabilidad: valores mayores permiten una identificación más rápida de la planta, mientras que valores menores producen un proceso de adaptación más lento, aunque igualmente estable.

Asimismo, se comprobó la importancia de la señal de excitación en el proceso adaptativo. Cuando se empleó una señal pseudoaleatoria, el filtro logró identificar correctamente la respuesta al impulso de la planta, alcanzando errores de identificación próximos al límite de precisión numérica de la implementación. En contraste, al utilizar una señal constante el algoritmo no logró converger hacia la solución correcta, verificando experimentalmente que una señal persistentemente excitante constituye una condición necesaria para la correcta identificación de sistemas mediante el algoritmo LMS.

A partir de estos resultados se validó la implementación desarrollada y se corroboró el comportamiento esperado del algoritmo LMS, en concordancia con los fundamentos teóricos estudiados durante el curso.

Como segunda etapa del trabajo, se implementó exitosamente un sistema de cancelación adaptiva de ruido sobre el microcontrolador STM32F446RE utilizando un filtro LMS. El sistema:

- Genera ruido blanco pseudoaleatorio de forma determinística mediante LCG.
- Modela la contaminación del ruido mediante un FIR parametrizable.
- Implementa el algoritmo LMS con convergencia predecible y estable.
- Procesa señales en tiempo real sin sobrecargar el procesador ($<10\%$ CPU).
- Proporciona dos canales de salida para monitoreo simultáneo de la señal limpia y la señal ruidosa.

La arquitectura basada en DMA y doble buffering permite mantener una operación continua con baja latencia. El algoritmo LMS demostró convergencia dentro de los tiempos esperados (aproximadamente 9 segundos), confirmando la viabilidad del enfoque para implementar sistemas de cancelación adaptativa de ruido sobre plataformas embebidas.

Como trabajo futuro, podrían incorporarse técnicas de ajuste automático del parámetro de adaptación μ , la implementación de variantes del algoritmo LMS, como NLMS o RLS, y la utilización del periférico RNG por hardware del microcontrolador para mejorar la calidad estadística de la señal pseudoaleatoria generada.